

Methoden benutzen

Instanzvariablen – Teil 1

Argumente und Rückgabewerte

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

Prof. Dr. Martin Burger { 17. November 2025 }

noan



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan 
überlebt den
ersten
Kontakt  mit
der Realität .

Frei nach Helmuth Karl
Bernhard von Moltke

Methoden benutzen Instanzvariablen

 Plan für diese Woche

 Objekte wissen und tun

 Methodenparameter

 Rückgabewerte

 Pass-by-Value

 Pair Programming

 Konventionen

 Getter und Setter

 Kapselung

 Variablen initialisieren

 Variablen vergleichen

 Pair Programming

 Spaß

📖 Methoden benutzen

🔗 Instanzvariablen

Verbindungen herstellen

👤 Tauschen Sie sich mit einer Nachbar:in aus!

🐱 Wie werden in Java Wissen und Verhalten repräsentiert?

➡ Wie können Informationen an eine Methode weitergegeben werden?

➡ Wie kann man Informationen von einer Methode erhalten?

💶 Was wird genau bei der Übergabe von Argumenten an Methoden übergeben?

🙋 Welche Fragen haben Sie zu diesem Kapitel?

4 Methoden benutzen Instanzvariablen

Wie sich Objekte verhalten



Zustand beeinflusst Verhalten, Verhalten beeinflusst Zustände. Wir wissen, dass Objekte über **Zustand** und **Verhalten** verfügen, die durch **Instanzvariablen** und **Methoden** repräsentiert werden. Dabei haben wir uns noch nicht angesehen, welche *Beziehung* Zustand und Verhalten zueinander haben. Wir wissen bereits, dass jede Instanz einer Klasse (jedes Objekt eines bestimmten Typs) seine eigenen, einmaligen Werte für Instanzvariablen besitzen kann. Hund (Dog-Objekt) A kann einen *Namen* haben, »Fido«, und ein *Gewicht* von 35 Kilo. Hund B heißt »Killer« und wiegt 5 Kilo. Wenn nun die Dog-Klasse eine Methode namens `makeNoise()` (`gibLaut()`) besitzt, sollte das Bellen eines 35-kg-Hunds dann nicht etwas tiefer sein als das seines deutlich kleineren Artgenossen? (Wobei wir davon ausgehen, dass ein nerviges Kläffen tatsächlich als *Bellen* durchgeht.) Glücklicherweise ist genau das der springende Punkt eines Objekts – es besitzt *Verhalten*, das sich auf seinen *Zustand* auswirkt. Anders gesagt, **Methoden benutzen die Werte von Instanzvariablen**. Ein Beispiel: »Wenn der Hund weniger als 7 Kilo wiegt, erzeuge ein Kläffgeräusch, sonst ...« oder »erhöhe das Gewicht um 5«. **Also los, lassen Sie uns ein paar Zustände ändern.**

Hier fängt ein neues Kapitel an 71



Objekte wissen und tun

Eine Klasse beschreibt, was ein Objekt weiß und tut

🐱 Objekte wissen und tun

- Objekte haben einen **Zustand** und ein **Verhalten**. Diese werden repräsentiert durch:
 - Instanzvariablen
 - Methoden
- Ein Objekt kann **verschiedene Werte** für **Instanzvariablen** haben.
- Jede Instanz einer Klasse hat die **gleichen Methoden**.
- **Methoden** können sich in Abhängigkeit von den Werten der Instanzvariablen **unterschiedlich verhalten**.



Instanzvariablen
(Zustand)

Methoden
(Verhalten)



weiß

tut

**Zustand beeinflusst Verhalten,
Verhalten beeinflusst Zustände.**



🐱 Objekte wissen und tun

Fünf Instanzen
der Klasse Song.



Unterschiedlicher
Zustand
(Instanzvariablen)

Unterschiedliches
Verhalten
(Methoden)

```
1 import java.util.*;
2
3 public class OddCatTestDrive {
4     public static void main(String[] args) {
5
6         OddCat cat = new OddCat();
7
8         System.out.println("Cat hasn't eaten all day...");
9         cat.hungry = true;
10        System.out.println("-> This starving cat is acting suspiciously predictable.");
11        for (int counter = 0; counter < 10; counter++) {
12            cat.act();
13        }
14
15        System.out.println("Cat just ate the whole can...");
16        cat.hungry = false;
17        System.out.println("-> I call that shenanigans! The fed cat is acting just as predictably.");
18        for (int counter = 0; counter < 10; counter++) {
19            cat.act();
20        }
21    }
22 }
23
```

STDIN

Input for the program (Optional)

Output:

[illegible]

```
1 import java.util.*;
2
3 public class CatTestDrive {
4     public static void main(String[] args) {
5
6         Cat cat = new Cat();
7
8         System.out.println("Cat hasn't eaten all day...");
9         cat.hungry = true;
10        System.out.println("-> This starving cat is acting as strangely as expected:");
11        for (int counter = 0; counter < 10; counter++) {
12            cat.act();
13        }
14
15        System.out.println("Cat just ate the whole can...");
16        cat.hungry = false;
17        System.out.println("-> All's right with the world! The fed cat is acting just as strangely:");
18        for (int counter = 0; counter < 10; counter++) {
19            cat.act();
20        }
21    }
22 }
23
```

STDIN

Input for the program (Optional)

Output:

```
Cat hasn't eaten all day...
-> This starving cat is acting as strangely as expected:
[hungry      <->] Purring and snuggling...
[hungry      <->] Purring and snuggling...
[bad-tempered <->] Hissing and clawing...
[bad-tempered <->] Hissing and clawing...
[bad-tempered <->] Hissing and clawing...
[bad-tempered <->] Hissing and clawing...
[easy-going  <->] Purring and snuggling...
[bad-tempered <->] Hissing and clawing...
[easy-going  <->] Purring and snuggling...
[easy-going  <->] Purring and snuggling...
Cat just ate the whole can...
-> All's right with the world! The fed cat is acting just as strangely:
[bad-tempered <->] Hissing and clawing...
[easy-going  <->] Purring and snuggling...
[bad-tempered <->] Hissing and clawing...
[bad-tempered <->] Hissing and clawing...
[bad-tempered <->] Hissing and clawing...
[easy-going  <->] Purring and snuggling...
[stuffed     <->] Hissing and clawing...
[stuffed     <->] Hissing and clawing...
[easy-going  <->] Purring and snuggling...
[easy-going  <->] Purring and snuggling...
```


! ? Quiz

🐱 Objekte wissen und tun

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



Klassen definieren, was ein Objekt weiß
und was es tut.

Richtig!

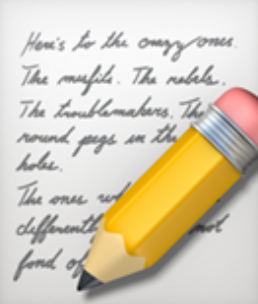
Die Dinge, die ein Objekt tut, werden Instanzvariablen genannt (Zustand).

Die Dinge, die ein Objekt weiß, sind seine Methoden (Verhalten).

Wissen → Zustand → Instanzvariablen. Tun → Verhalten → Methoden.

Methoden verwenden Instanzvariablen und können daher Instanzen des gleichen Typs zu unterschiedlichem Verhalten veranlassen.

Richtig!

 Welcher **eine** Gedanke zu
„ Objekte wissen und tun“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

Methodenparameter

Informationen übergeben

➡ Methodenparameter

- Sie können Methoden **Werte** übergeben.
 - `song.repeat(3)`
- Argumente oder Parameter?   **Diskussion bei StackOverflow**
 - Ein **Aufrufer** übergibt konkrete **Werte** als **Argumente**.
 - Eine **Methode** definiert und übernimmt **Parameter**.
 - Der Parameter wird zur **Variablen in der Methode**, deren Typ und Name im Methodenkörper verwendet werden kann.
- Wenn eine Methode **Parameter** definiert, muss beim Aufruf etwas **übergeben** werden.
 - Und zwar vom **richtigen Typ**.



Ruft die hiss-Methode auf
und übergibt den Wert 2.
(als Argument der Methode)

```
Cat c = new Cat();  
c.hiss(2);
```

Die Bits landen im
Parameter hisses.
(Variable mit int-Größe)

Die Bits, die 2
repräsentieren,
werden übergeben.
(Literaler Wert vom Typ int)

```
void hiss(int hisses) {  
    while (hisses > 0) {  
        System.out.println("Hiss!");  
        hisses = hisses - 1;  
    }  
}
```

Verwendet den Parameter als
Variable im Methodenkörper.
(Name und Typ)

Mehrere Argumente

➡ Methodenparameter

- Methoden können **mehrere Parameter** haben.
- Diese werden bei der Deklaration durch **Kommata** getrennt.
- Ebenso werden die Argumente bei der Übergabe durch Kommata getrennt.
- Wenn eine Methode Parameter hat, müssen die Argumente
 - den richtigen Typ und
 - die **richtige Reihenfolge** haben.



! ? Quiz

➡ Methodenparameter

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Eine Methode kann Parameter haben. Das bedeutet, dass Sie der Methode einen oder mehrere Werte als Argumente übergeben können.

Richtig!

Die Reihenfolge der an eine Methode übergebenen Argumente spielt keine Rolle. Der Compiler bringt die Argumente entsprechend ihrer Typen in die richtige Reihenfolge.

Argumente müssen mit den Typen und mit der Reihenfolge der deklarierten Parameter übereinstimmen.

Sie können einer Methode neben literalen Werten (wie 2, ' c ') auch Variablen übergeben (zum Beispiel s, wobei s eine `String`-Variable ist), solange der Typ der Variablen mit dem Typ des entsprechenden Parameters übereinstimmt.

Richtig!

 Welcher **eine** Gedanke zu
„ Methodenparameter“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!



Durchatmen und  Sortieren

Rückgabewerte

Informationen erhalten

➡ Rückgabewerte

- Methoden können **Rückgabewerte** haben.
- Jede Methode wird mit einem **Rückgabotyp** deklariert.
- Gibt eine Methode nichts zurück: `void`
- Ansonsten der Typ des Wertes, der zurückgegeben wird:
 - Elementartyp (`int`, `char`, ...)
 - Objektreferenz (`Song`, `Cat`, ...)



Diese Typen müssen passen!
(Rückgabotyp und Typ der Variablen)

```
int getEvenPrimeNumber() {  
    return 2;  
}
```

Dieser Wert muss vom
Rückgabotyp sein!
(Rückgabewert muss in Becher passen.)

```
int theOnlyOne = eratosthenes.getEvenPrimeNumber();
```

Der von der Methode zurückgegebene
Wert wird in der Variablen gespeichert.
(Die Bits, die diesen Wert repräsentieren.)

! Quiz

➡ Rückgabewerte

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!

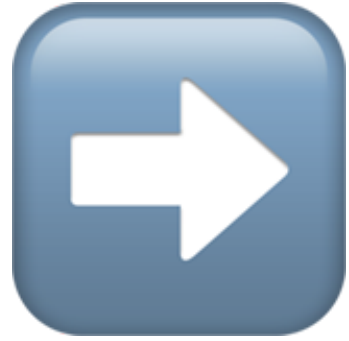


Eine Methode muss einen Rückgabotyp deklarieren. Der Rückgabotyp `null` bedeutet, dass die Methode nichts zurückgibt.

Fast! Dies ist der Rückgabotyp `void`.

Wenn eine Methode einen Rückgabotyp deklariert, der nicht `void` ist, muss der Typ des Rückgabewerts zum deklarierten Rückgabotyp passen.

Richtig!

 Welcher **eine** Gedanke zu
„ Rückgabewerte“ war für
Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren



Pass-by-Value

„Pass-by-Copy“



Pass-by-Value

- Java verwendet Pass-by-Value.
 - Dabei wird der **Wert** einer Variablen übergeben.
 - Die **Bits** (primitiver Wert oder Referenz) werden **kopiert**.
- Auf diese Weise können Methoden den Wert (die Bits) in der **aufrufenden** Variablen **nicht verändern**.
- Daher funktioniert das Beispiel `hiss(int hisses)` ohne „überraschende Nebeneffekte“.




```
Cat c = new Cat();  
c.hiss(2);
```

```
void hiss(int hisses) {  
    while (hisses > 0) {  
        System.out.println("Hiss!");  
        hisses = hisses - 1;  
    }  
}
```

```
Cat myCat = new Cat();  
hiss2Times = 2;  
myCat.hiss(hiss2Times);  
System.out.print(hiss2Times);
```

```
void hiss(int hisses) {  
    while (hisses > 0) {  
        System.out.println("Hiss!");  
        hisses = hisses - 1;  
    }  
}
```


„Pass-by-Copy“



Pass-by-Value

- Java verwendet Pass-by-Value.
 - Dabei wird der **Wert** einer Variablen übergeben.
 - Die **Bits** (primitiver Wert oder Referenz) werden **kopiert**.
- Auf diese Weise können Methoden den Wert (die Bits) in der **aufrufenden** Variablen **nicht verändern**.
- Daher funktioniert das Beispiel `hiss(int hisses)` ohne „überraschende Nebeneffekte“.



Pass-by-Reference



Pass-by-Value

- Andere Sprachen wie C:
Pass-by-Reference .
- Die **Referenz auf eine Variable** („Pointer“/„Zeiger“) wird übergeben.
- Dadurch kann die Methode den Inhalt in der übergebenen Variablen ändern.



```
Cat myCat = new Cat();  
myCat.hungry = true;  
game.hiss(myCat, 2);  
System.out.print(myCat.hungry);
```

```
void hiss(Cat cat, int hisses) {  
    while (hisses > 0 && cat.hungry) {  
        System.out.println("Hiss!");  
        hisses = hisses - 1;  
    }  
    cat.hungry = false; // What could possibly go wrong?  
}
```


! ? Quiz



Pass-by-Value

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Nimmt eine Methode zum Beispiel ein Argument `counter` vom Typ `int` entgegen und ändert den Wert von `counter` innerhalb der Methode, so erhält der aufrufende Code automatisch den neuen Wert in `counter`.

Java verwendet Pass-by-Value. Die Methode kann daher den Wert nach außen nicht ändern.

Bei Pass-by-Value wird eine Kopie des Variablenwerts an eine Methode übergeben, nicht die Variable selbst.

Richtig!

Bei der Übergabe von Objekten an Methoden wird eine Kopie der Referenz auf das Objekt übergeben, nicht das Objekt selbst.

Richtig!

Änderungen an den Argumenten innerhalb der Methode wirken sich daher nicht auf die ursprünglichen Variablen außerhalb der Methode aus, es sei denn, die Eigenschaften eines Objekts werden über die kopierte Referenz geändert.

Richtig!

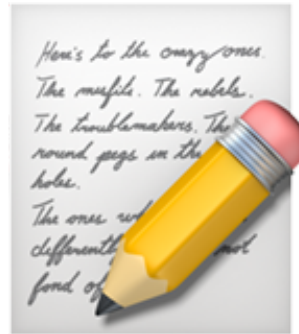


Welcher **eine** Gedanke zu



„Pass-by-Value“ war für

Sie am wichtigsten?



Schreiben Sie ihn auf!



Durchatmen und



Sortieren

Pair Programming

🎓 **Argumente und Rückgabewerte**

🤝 **Pair Programming**

🎯 **Implementieren Sie einen einfachen Taschenrechner!**

✅ Zustand: an/aus

✅ Verhalten: Addition ($a + b$)

✅ wenn an: gibt die Summe der Argumente zurück

✅ wenn aus: gibt immer 0 zurück



🎓 Argumente und Rückgabewerte

🤝 Pair Programming

🎯 Implementieren Sie einen einfachen Taschenrechner!

- ✅ Zustand: an/aus
- ✅ Verhalten: Addition ($a + b$)
 - ✅ wenn an: gibt die Summe der Argumente zurück
 - ✅ wenn aus: gibt immer 0 zurück

- ✅ Verwenden Sie geeignete Typen für Instanzvariable, Parameter und Rückgabewert.

🚩 Am Ende sollten Sie eine Klasse `Calculator`

🚩 Attribut: `switchedOn`

🚩 Methode: `add(...)`

🚩 ... und eine Testklasse `CalculatorTestDrive` haben.


```
1 import java.util.*;
2
3 public class CalculatorTestDrive {
4     public static void main(String[] args) {
5
6         Calculator calc = new Calculator();
7
8         System.out.println("-> Turn on the calculator and add some numbers:");
9         calc.switchedOn = true;
10        System.out.println("10 + 20 -> " + calc.add(10, 20));
11        System.out.println("-1 + -2 -> " + calc.add(-1, -2));
12
13        System.out.println("-> Turn off the calculator and add some numbers:");
14        calc.switchedOn = false;
15        System.out.println("10 + 20 -> " + calc.add(10, 20));
16        System.out.println("-1 + -2 -> " + calc.add(-1, -2));
17    }
18 }
19
```

STDIN

Input for the program (Optional)

Output:

-> Turn on the calculator and add some numbers:

10 + 20 -> 30

-1 + -2 -> -3

-> Turn off the calculator and add some numbers:

10 + 20 -> 0

-1 + -2 -> 0

 Welcher **eine** Gedanke zu
„ Argumente und
Rückgabewerte“ war für Sie am
wichtigsten?  Schreiben Sie ihn
auf!

 Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Methoden benutzen Instanzvariablen

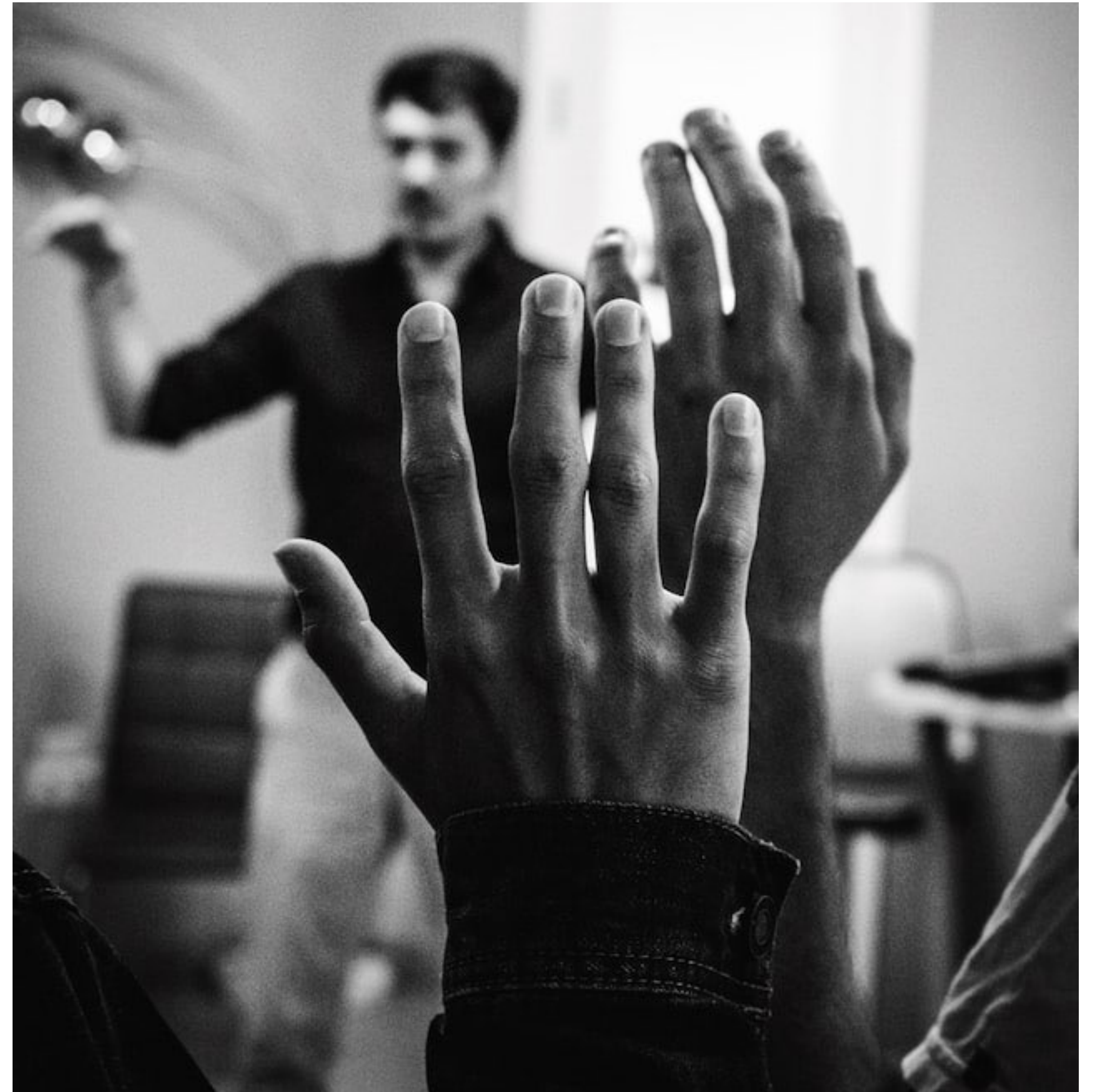
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Methoden benutzen Instanzvariablen?**
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Methoden benutzen Instanzvariablen



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

